GeoServer, dan antarmuka WebGIS berbasis React.js. Rincian kebutuhan non-fungsional disajikan pada Tabel 5.

Tabel 5. Kebutuhan Non-Fungsional Sistem

Kode	Aspek	Pernyataan Kebutuhan
NF-01	Keamanan	Sistem membatasi akses data e-BPHTB bersifat *read-only* untuk menjaga integritas data sumber
NF-02	Performa	Sistem merespons permintaan layanan GeoServer dalam waktu yang wajar
NF-03	Akurasi / Reliabilitas	Sistem mempertahankan konsistensi posisi geometri selama proses transformasi data dari Oracle Spatial ke WebGIS
NF-04	Pemeliharaan	Sistem tidak mengubah struktur tabel sumber pada proses integrasi data
NF-05	Pemeliharaan	Sistem tidak mengubah opsi *layer* tematik yang sudah ada sebelumnya (Status Pembayaran SPPT)

4.2 Hasil Perancangan Sistem (*Design*)

Berdasarkan hasil identifikasi kebutuhan pada subbab 4.1, tahap perancangan sistem dituangkan ke dalam *use case diagram*, *activity diagram*, *sequence diagram*, dan *class diagram*.

Sebelum masuk ke diagram, ditetapkan skema klasifikasi warna yang menjadi dasar logika pewarnaan pada seluruh diagram berikut, disajikan pada Tabel 6.

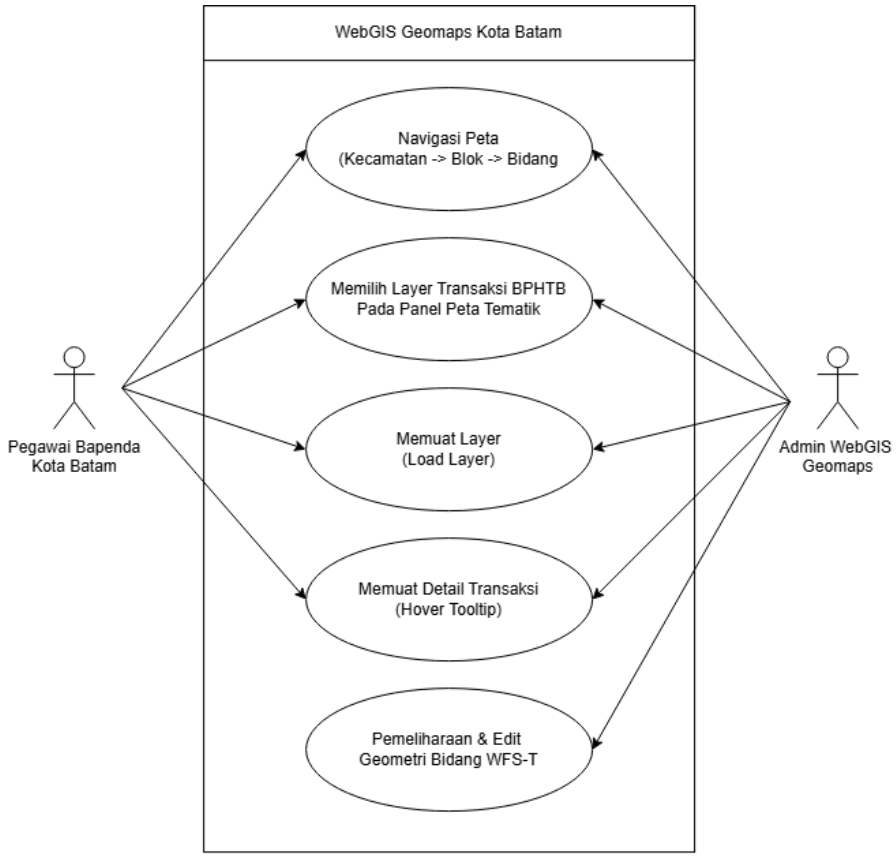
Tabel 6. Klasifikasi Kategori Nilai Transaksi

Kategori	Rentang Nilai Transaksi	Warna
Nilai Rendah	≤ Rp 100.000.000	Hijau
Nilai Sedang	Rp 100.000.001 – Rp 500.000.000	Oranye
Nilai Tinggi	> Rp 500.000.000	Merah

4.2.1 *Use Case Diagram*

Berdasarkan hasil analisis dibuat *use case diagram* yang terdiri dari 2 Peran Aktor. Untuk Aktor Pegawai Bapenda dapat melakukan proses Navigasi Peta mulai dari pilih kecamatan, lalu pilih kelurahan yang ada di kecamatan, lalu pilih blok dan terakhir bisa melihat bidang pada area blok tersebut. Aktor juga dapat memilih *layer* transaksi pada panel peta tematik, memuat *layer*, dan melihat detail transaksi melalui hover tooltip. Untuk Aktor Admin dapat melakukan aksi yang sama seperti Aktor Pegawai Bapenda, dengan tambahan dapat membuat dan mengedit polygon baru pada bidang WebGIS.

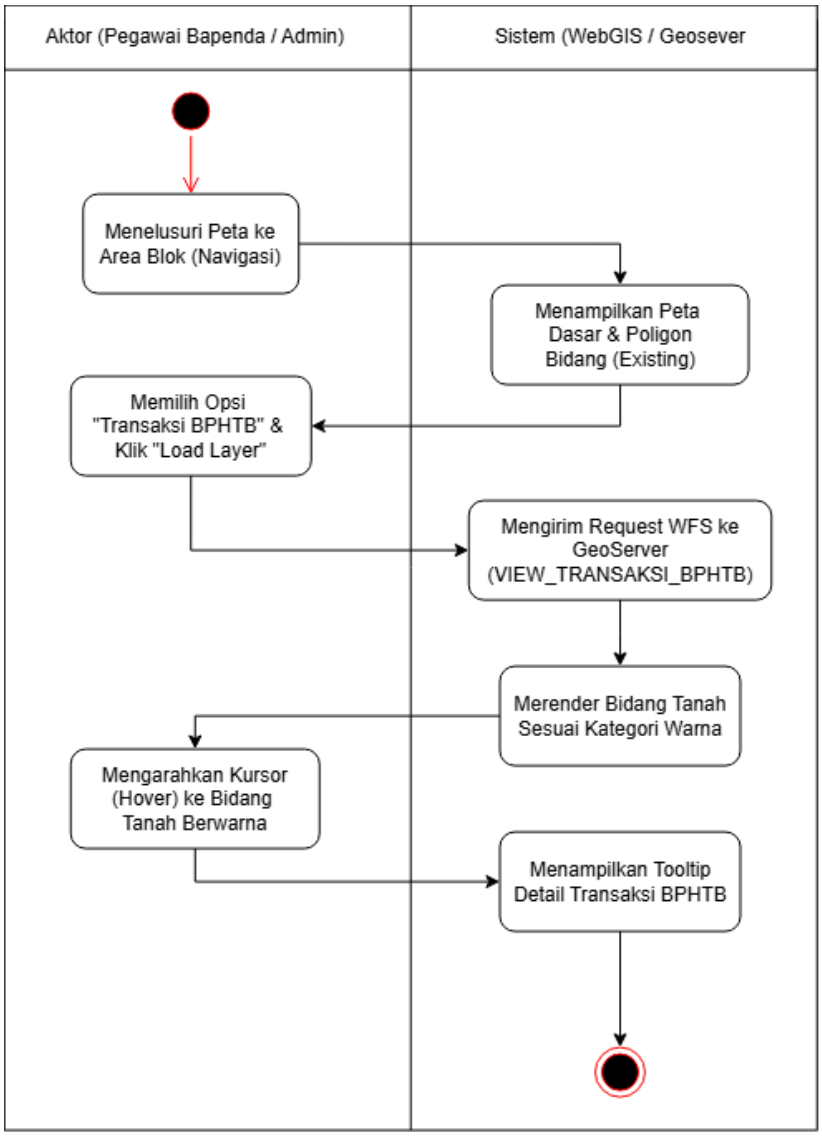
Hasil rancangan *use case diagram* dapat dilihat pada Gambar 4.



Gambar 4. *Use Case Diagram*

4.2.2 *Activity Diagram*

Activity diagram berikut menggambarkan alur menampilkan peta tematik BPHTB, mulai dari navigasi hingga interaksi *hover*. Hasil perancangan *activity diagram* dapat dilihat pada Gambar 5.



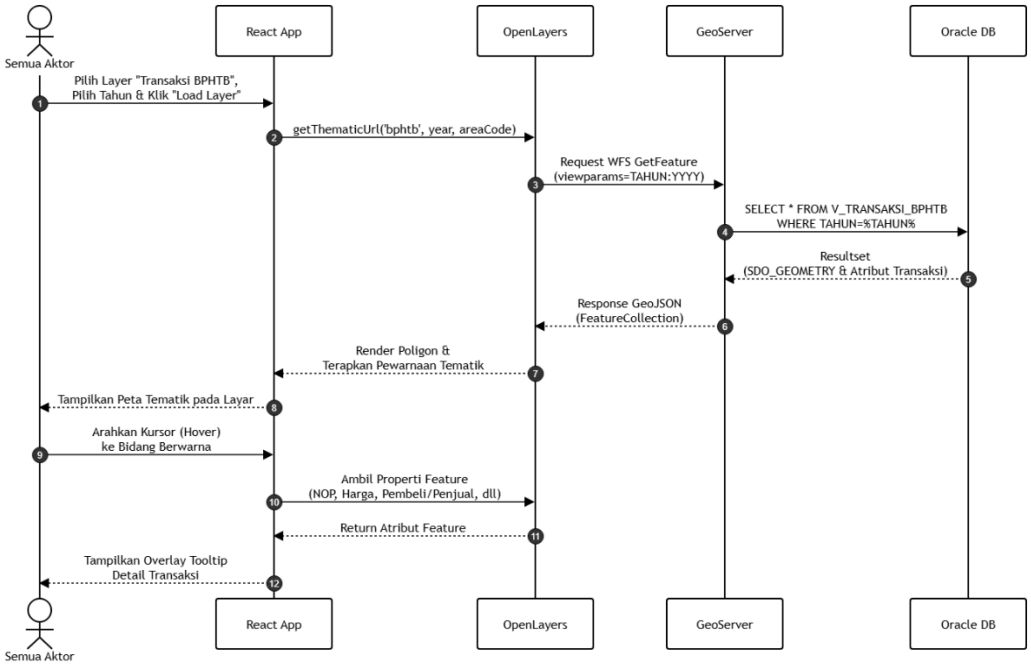
Gambar 5. *Activity Diagram*

Hasil rancangan dari *activity diagram* pada penelitian menggambarkan proses action yang dilakukan oleh aktor dari awal hingga akhir proses. Alur dimulai ketika

aktor melakukan proses navigasi dari memilih kecamatan, memilih kelurahan, memilih blok, lalu kemudian dari sistem akan melakukan proses load map bidang dan polygon sesuai dengan level administratif wilayah. Setelah pada bidang wilayah yang menampilkan polygon tiap bidang (level administratif wilayah blok), aktor mengaktifkan *layer* peta tematik transaksi BPHTB, memilih filter tahun dan klik "*load layer*". Sistem akan mengirimkan request ke geoserver untuk menjalankan query VIEW_TRANSAKSI_BPHTB sesuai dengan parameter tahun yang dipilih user juga dan kemudian dikirim melalui function endpoint di kode.

4.2.3 *Sequence Diagram*

Sequence diagram menggambarkan urutan interaksi antar komponen sistem untuk proses menampilkan peta tematik. Hasil perancangan *sequence diagram* pada penelitian ini dapat dilihat pada Gambar 6.



Gambar 6. *Sequence Diagram*

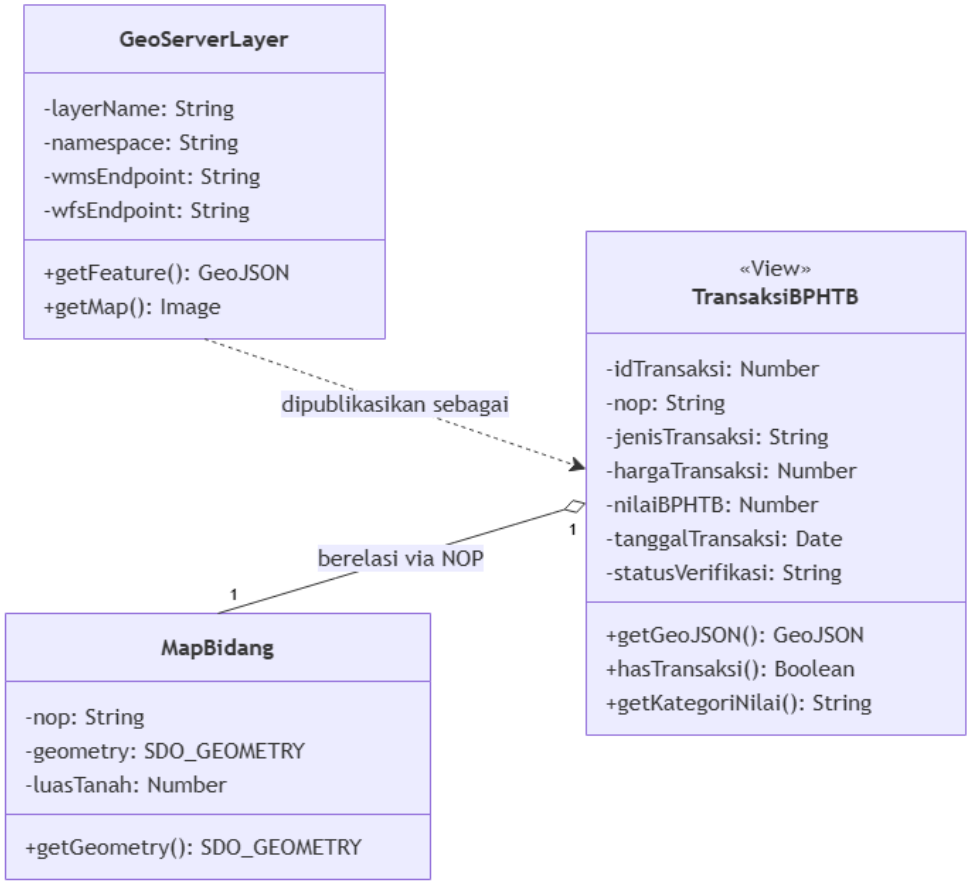
Gambar 6 menampilkan rancangan *sequence diagram* yang menggambarkan alur interaksi antarkomponen dalam memuat *layer* peta tematik Transaksi BPHTB. Proses dirancang diawali ketika pengguna memilih opsi *layer* "Transaksi BPHTB" dan menentukan tahun transaksi pada panel Peta Tematik, lalu mengeklik tombol "Load *Layer*". *React App* merespons interaksi tersebut dengan memanggil fungsi getThematicUrl('bphtb', year, areaCode) untuk menyusun URL permintaan secara dinamis. Selanjutnya, OpenLayers mengirimkan *request* WFS GetFeature ke GeoServer, yang kemudian dieksekusi sebagai *query* SQL SELECT * FROM VIEW_TRANSAKSI_BPHTB pada basis data Oracle DB.

Tahap berikutnya menggambarkan alur pengembalian data dan pemrosesan visual pada antarmuka. Oracle DB mengembalikan *resultset* data spasial (SDO_GEOMETRY) beserta atribut transaksi e-BPHTB ke GeoServer untuk dikonversi menjadi format GeoJSON. OpenLayers bersama *React App* menerima GeoJSON tersebut, lalu merender poligon bidang tanah dan menerapkan logika pewarnaan dinamis sesuai kategori nilai transaksi (Hijau, Oranye, Merah). Rancangan interaksi diakhiri dengan pemrosesan interaktif, di mana saat pengguna mengarahkan kursor (*hover*) ke bidang tanah berwarna, *React App* mengambil properti *feature* dari OpenLayers untuk menampilkan *tooltip* detail transaksi secara *real-time*.

4.2.4 *Class Diagram*

Class diagram berikut menggambarkan entitas konseptual utama beserta atributnya. Diagram ini merepresentasikan entitas data dan relasinya, bukan kelas

pemrograman, karena sistem berbasis integrasi database view. Hasil perancangan *class diagram* dapat dilihat pada Gambar 7.



Gambar 7. *Class Diagram*

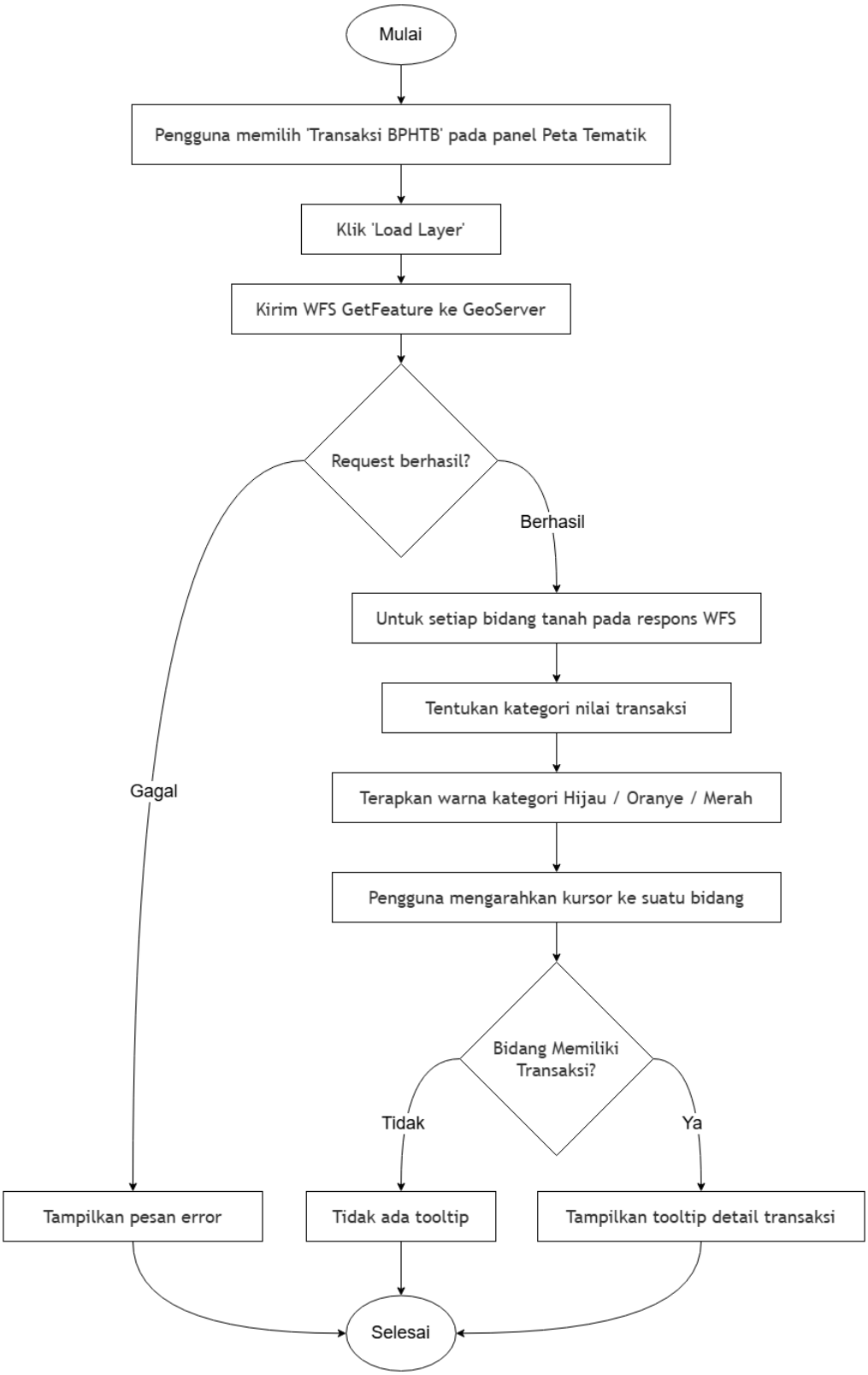
Kelas TransaksiBPHTB dengan stereotip <<View>> dirancang untuk memetakan hasil penggabungan data spasial dan transaksional pada tingkat basis data. Kelas ini menyimpan atribut seperti idTransaksi, nop, jenisTransaksi, hargaTransaksi, nilaiBPHTB, tanggalTransaksi, dan statusVerifikasi, serta dilengkapi metode getGeoJSON(), hasTransaksi(), dan getKategoriNilai(). Sementara itu, kelas MapBidang merepresentasikan objek spasial bidang tanah di

lapangan yang menyimpan atribut nop, geometry (SDO_GEOMETRY), dan luasTanah.

Kelas GeoServerLayer dirancang untuk mengelola pemublikasian layanan geospasial yang menyimpan konfigurasi *endpoint* WMS/WFS, *namespace*, dan nama *layer*, dengan metode utama getFeature() dan getMap(). Relasi antar-kelas menunjukkan bahwa satu objek TransaksiBPHTB terhubung secara opsional dengan satu objek MapBidang berbasis kunci relasi NOP. Kelas GeoServerLayer juga memiliki relasi ketergantungan (*dependency*) terhadap TransaksiBPHTB karena bertugas mempublikasikan *view* basis data tersebut menjadi layanan *web feature service* yang siap dikonsumsi oleh *frontend*.

4.2.5 *Flowchart* Proses Kerja Sistem

Flowchart berikut menggambarkan alur kerja sistem secara teknis, termasuk penanganan kondisi permintaan layanan yang gagal. *Flowchart* ini berisi rangkaian prosedural untuk memahami alur sistem, dan digunakan sebagai *flowchart* program / algoritma. Hasil perancangan *flowchart* proses kerja sistem dapat dilihat pada Gambar 8.



Gambar 8. *Flowchart* Proses Kerja Sistem